Learning to Flip: Reinforcement Learning for Stochastic Local Search

Prabhav Singh

The Boolean Satisfiability Problem (SAT) is the canonical NP-complete problem. Stochastic Local Search (SLS) solvers, of which WalkSAT [7] is an example, are among the most common approaches for satisfiable instances. They maintain a complete variable assignment and iteratively *flip* variable values to reduce the number of violated clauses. Their central design choice is the *flip heuristic*: given a randomly chosen unsatisfied clause, which variable in that clause should be flipped?

Classical heuristics like *min-break*[1] and Novelty+ [6] are hand-designed and distribution-agnostic. Yet real-world SAT instances arising from a common structural source like scheduling encodings, planning problems, hardware benchmarks, share deep regularity. This motivates a natural question: can a flip heuristic be learned from a family of such instances and **generalize to new, unseen instances** from the same family?

Related Work

Several recent works apply machine learning to SAT solving. NeuroBack [8] and similar methods target CDCL solvers via phase or branching heuristics. NLocalSAT [10] uses a GNN to warm-start SLS initialization, but leaves the per-step flip heuristic unchanged[2].

Within the SLS-specific learning literature, Yolcu & Póczos [9] apply RL with a full GNN to score flip candidates. The GNN is expressive but expensive per step, restricting evaluation to tiny instances and showing limited gains on random SAT. Interian & Bernardini [4] improve substantially with a fast *linear* scoring function over local hand-crafted features,[3] trained via REINFORCE on a fixed instance distribution. This model outperforms both WalkSAT and the GNN approach. In the broader combinatorial optimization literature, instance-specific *online adaptation*—fine-tuning a policy during test-time on the target instance itself—has been shown to consistently improve over frozen models [1, 3]. However, this paradigm has not been applied to learned flip heuristics for SLS. **Our contribution** extends this line in three ways:

1. We replace the linear scorer with a small MLP, allowing the model to capture non-linear feature interactions[4]. The linear model is a strict special case; whether the added capacity helps on structured families is an open empirical question with a clean answer.
2. **Shorter-horizon returns.** Interian's REINFORCE uses full-episode returns, creating a credit assignment problem: early flips receive highly noisy signals. We instead plan to use k-step discounted returns, tightening reward attribution to the flips most responsible for local progress.
3. **Online instance-specific adaptation.** We explore interleaving gradient updates with flip decisions on a single target instance. This is structurally distinct from active search in routing problems (which updates between solution constructions); here, updates occur *during* the SLS solve trajectory, making credit assignment and stability the central challenges.

Method Plan

MDP Formulation. We propose to model each SLS trajectory as an episodic MDP $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where:

- $\mathcal{S}$: the state space, where each state $s_t = (\alpha_t, h_t)$ consists of the current variable assignment $\alpha_t \in \{0,1\}^n$ over n Boolean variables, and a history map $h_t : X \to \mathbb{Z}_{\geq 0}$ recording the last step at which each variable was flipped.
- $\mathcal{A}$: the action space, where at each step a random unsatisfied clause c_t is drawn uniformly and the action $a_t \in \text{vars}(c_t)$ selects which variable in that clause to flip.
- T: the (deterministic) transition $\alpha_{t+1} = \alpha_t \oplus e_{a_t}$, flipping exactly the chosen variable.

[1] Flipping the variable that breaks fewest currently satisfied clauses.

[2] In more detail, they use a LSTM architecture to choose appropriate assignments of variables **before solving** to initialize the SAT-solver better.

[3] Features include break count and two binary recency indicators (flipped in last 5 or 10 steps).

[4] A variable that is simultaneously high-break *and* recently flipped may warrant disproportionate avoidance in ways a linear model cannot represent.

- R: the reward $r_t = \text{make}(a_t) - \text{break}(a_t)$, the net change in satisfied clauses after the flip, where $\text{make}(x)$ and $\text{break}(x)$ count clauses newly satisfied or violated by flipping x[5].
- γ: a discount factor applied over a k-step return $G_t = \sum_{i=0}^{k-1} \gamma^i r_{t+i}$, chosen to tighten credit assignment relative to full-episode returns used in prior work [4].

Policy. For each candidate variable $x \in \text{vars}(c_t)$, we plan to compute a local feature vector $\phi(x, s_t)$ encoding statistics such as break count and recency of flipping (all computable in $O(1)$ per variable per step). A learned scoring function f_θ (an MLP) maps these features to a scalar score; the policy then samples proportionally to the softmax over candidates in c_t. We intend to train θ via REINFORCE with mean-baseline variance reduction on batches of instances from a target family, comparing against a linear f_θ as an ablation[6].

Experimental Plan

We evaluate on two definite instance families: **random 3-SAT** at the phase transition[7] and **graph k-coloring** encoded as SAT via CNFgen [5] (structured, with prior results for comparison). We additionally hope to *explore* SAT encodings of scheduling problems as a secondary domain. Baselines are WalkSAT min-break, Novelty+, and a linear scorer replicating Interian et al. We would report median flips-to-solution, solve rate, and runtime distribution curves. Key hypotheses: **(H1)** MLP > linear on structured families; **(H2)** no improvement on random 3-SAT; **(H3)** no cross-family transfer of learned heuristics.

References

[1] Irwan Bello, Hieu Pham, Quoc V Le, Mohammad Norouzi, and Samy Bengio. Neural combinatorial optimization with reinforcement learning. In *International Conference on Learning Representations (Workshop)*, 2017.

[2] Holger H Hoos. An adaptive noise mechanism for WalkSAT. In *Proceedings of the Eighteenth National Conference on Artificial Intelligence (AAAI-02)*, pages 655–660, 2002.

[3] André Hottung, Yeong-Dae Kwon, and Kevin Tierney. Efficient active search for combinatorial optimization problems. In *International Conference on Learning Representations (ICLR)*, 2022.

[4] Yannet Interian and Sara Bernardini. Learning interpretable heuristics for WalkSAT. In *Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 365–374, 2023.

[5] Massimo Lauria, Marc Olivecrona, and Jakob Nordström. CNFgen: A generator of crafted benchmarks. In *Proceedings of the 20th International Conference on Theory and Applications of Satisfiability Testing (SAT)*, pages 464–473, 2017.

[6] David McAllester, Bart Selman, and Henry Kautz. Evidence for invariants in local search. In *Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97)*, pages 321–326, 1997.

[7] Bart Selman, Henry Kautz, and Bram Cohen. Noise strategies for improving local search. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI-94)*, pages 337–343, 1994.

[8] Wenxi Wang, Yang Yang, Mohit Subramanian, and Moshe Y Vardi. NeuroBack: Improving CDCL SAT solving using graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2024.

[9] Emre Yolcu and Barnabás Póczos. Learning local search heuristics for boolean satisfiability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[10] Wenjie Zhang, Zeyu Sun, Qihao Zhu, Ge Li, Shaowei Cai, Yingfei Xiong, and Lu Zhang. NLocalSAT: Boosting local search with solution prediction. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1596–1602, 2020.

[5] This local reward is a natural starting point but has limitations: it provides no signal during stagnation and does not account for clause difficulty. We plan to explore alternatives, including a **stagnation penalty** that subtracts a term proportional to the number of steps since the last improvement, and a clause-weighted reward that upweights satisfaction of frequently violated clauses; analogous to the SAPS/PAWS family of hand-designed adaptive schemes [2].

[6] The online variant is *not* meta-learning (we do not learn to learn across tasks) and differs from active search [1] (which re-optimizes between full solution constructions). It is closer to policy gradient fine-tuning interleaved with search: gradient steps are taken every Δ flips using recent trajectory segments as the mini-batch, with a small learning rate to prevent catastrophic drift away from the cross-instance initialization.

[7] An unstructured control where no advantage expected.